2. Nếu r=0 thì g:=n

Ngược lại (r>0) m:=n; n:=r và quay lại bước 1.

1.3. Sự ra đời và phát triển của ngôn ngữ C

Năm 1970 Ken Thompson sáng tạo ra ngôn ngữ B dùng trong môi trường hệ điều hành UNIX trên máy điện toán DEC PD-7. B cũng là chữ tắt của BCPL (Basic Combined Progamming Language) do Martin Richards viết. Năm 1972 Dennis Ritchie của hãng Bell Laboratories (và Ken Thompson) sáng tạo nên ngôn ngữ C nhằm tăng hiệu quả cho ngôn ngữ B. Lúc đầu ngôn ngữ C không được mọi người ưa dùng. Nhưng sau khi D.Ritchie cho xuất bản cuốn "The C Programming Language" ("Ngôn ngữ lập trình C") thì ngôn ngữ C được chú ý và được sử dụng rộng rãi. Người ta đã dùng C để viết hệ điều hành đa nhiệm UNIX, O/S 2 và ngôn ngữ Dbase. C đã được cải tiến qua nhiều phiên bản: trình biên dịch Turbo C từ phiên bản 1 đến phiên bản 5, Microsoft C từ phiên bản 1 đến phiên bản 6. Hiện nay, C lại được phát triển để thành C++ với 3 trình biên dịch: Borland C++, Visual C++ và Turbo C++.

Mặc dù hiện nay có khá nhiều ngôn ngữ lập trình mới, nhưng C vẫn là một ngôn ngữ lập trình được ưa chuộng. C được ứng dụng để viết các phần mềm trong nhiều lĩnh vực, đặc biệt là trong khoa học kỹ thuật.

2. MỘT SỐ KIẾN THỨC CƠ SỞ

2.1. Bộ kí tự, từ khóa,tên

2.1.1 Bộ kí tự trong C

Mọi ngôn ngữ đều được xây dựng trên một bộ kí tự (các chữ, các kí hiệu). Đối với ngôn ngữ C sử dụng bộ kí tự sau:

Tập các chữ cái in hoa: A, B, C, D, . ., Z

Tập các chữ cái in thường: a, b, c, d, . . , z

Tập các chữ số: 0, 1, 2, 3, . . , 9

Các dấu chấm câu: , . ; : / ? [ ] { } ! @ # $ ^ & * ( ) + = - < > "

Các kí tự không nhìn thấy: dấu trống (Space), dấu Tab, dấu xuống dòng (Enter),

Dấu gạch dưới _

2.1.2 Các từ khoá (Keywords)

Từ khoá là tập các từ dùng riêng của ngôn ngữ, mỗi từ khoá mang theo một ý nghĩa và tác dụng riêng. Từ khoá không thể định nghĩa lại và cũng không thể lấy từ khoá đặt tên cho các đối tượng. Dưới đây là bảng liệt kê các từ khoá thông dụng trong C.

auto	break	base	char	continue	default
do	double	else	extern	float	for

goto	if	int	long	register	return
short	sizeof	static	struct	switch	typedef
union	unsigned	void	public	while	volatile

2.1.3 Tên và cách đặt tên

Tên hay còn gọi là định danh (identifier) dùng để gọi các biến, hằng hoặc hàm. Đối với ngôn ngữ C, mọi tên phải được khai báo trước khi sử dụng. Tên là dãy các kí tự liền nhau gồm các chữ cái, a . . z, A. . Z, các chữ số 0. . 9 và dấu gạch dưới (dấu gạch dưới thường dùng để liên kết các thành phần của tên). Tuy nhiên, tên không được bắt đầu bằng chữ số và không chứa các kí tự đặc biệt như dấu cách, dấu tab và dấu chấm câu. Không được lấy từ khoá của C để đặt tên cho đối tượng.

Ví dụ về cách đặt tên đúng: Delta, E_Mu_X, Function1 . . .

Ví dụ về cách đặt tên sai:

2Delta: bắt đầu bằng kí tự số

E Mu_X: chứa khoảng trắng

Ngôn ngữ C phân biệt chữ in hoa và chữ in thường, do vậy những tên sau đây là khác nhau: x <> X, While <> while, For <> for. Do vậy, chúng ta cần lưu ý trong khi viết chương trình. Thông thường tên các biến, hàm được đặt bằng chữ in thường, tên các hằng được đặt bằng chữ in hoa.

2.1.4 Lời giải thích

Trong khi viết chương trình, đôi khi chúng ta cần ghi thêm một số lời ghi chú hoặc giải thích để chương trình trở nên dễ hiểu và dễ đọc. Lời giải thích không có tác dụng tạo nên mã chương trình và sẽ được trình dịch bỏ qua trong khi dịch chương trình. Phần ghi chú có thể biểu hiện trên nhiều dòng và được đánh dấu bởi cặp kí hiệu /* *đoạn văn bản ghi chú* */.

2.2. Cấu trúc chương trình trong C

2.2.1 Cấu trúc tổng quát của chương trình trong C

Chương trình tổng quát viết bằng ngôn ngữ C được chia thành 6 phần, trong đó có một số phần có thể có hoặc không có tuỳ thuộc vào nội dung chương trình và ý đồ của mỗi lập trình viên.

Phần 1: Khai báo các chỉ thị đầu tệp và định nghĩa các hằng sử dụng trong chương trình.

Phần 2: Định nghĩa các cấu trúc dữ liệu mới (user type) để sử dụng trong khi viết chương trình.

Phần 3: Khai báo các biến ngoài (biến toàn cục) được sử dụng trong chương trình.

Phần 4: Khai báo nguyên mẫu cho hàm (Function Ptototype). Nếu khai báo qui cách xây dựng và chuyền tham biến cho hàm, compiler sẽ tự động kiểm tra giữa nguyên mẫu

của hàm có phù hợp với phương thức xây dựng hàm hay không trong văn bản chương trình.

Phần 5: Mô tả chi tiết các hàm, các hàm được mô tả phải phù hợp với nguyên mẫu đã được khai báo cho hàm.

Phần 6: Hàm *main()*, hàm xác định điểm bắt đầu thực hiện chương trình và điểm kết thúc thực hiện chương trình.

2.2.2 Các bước cơ bản khi viết chương trình

Bước 1: Soạn thảo chương trình (dùng Turbo C)

Soạn thảo chương trình là giai đoạn dùng chương trình soạn thảo để viết văn bản chương trình. Turbo C trang bị một chương trình soạn thảo, dịch và thực hiện chương trình ngay trong môi trường của C, đó là chương trình có tên TC.EXE. Bản thân TC.EXE cũng trang bị cho người sử dụng một số phím chức năng giống như TURBO.EXE để soạn thảo. Khi ghi file văn bản chương trình lên đĩa, TC.EXE ngầm định đặt phần mở rộng của file là *.C mà ta thường gọi là chương trình nguồn (source program). Sau đây là một số phím chức năng cơ bản nhất của TC.EXE.

F1 (help) : Thực hiện chương trình trợ giúp trong khi viết chương trình.

CTRL + F1 : Thực hiện trợ giúp nhanh trong khi soạn thảo

F2 (save) : Ghi file văn bản chương trình lên đĩa với phần mở rộng là *.c

CTRL + F2 : Ghi file văn bản chương trình lên đĩa với một tên khác có phần mở rộng là *.c

F3 : Mở file mới để thực hiện soạn thảo.

Alt + F3 : Đóng file văn bản đang trong cửa sổ soạn thảo hiện thời

F4 : Dịch và thực hiện chương trình cho tới khi gặp dòng lệnh mà tại vị trí đó chúng ta bấm F4

F5 : Mở rộng hoặc thu nhỏ vùng soạn thảo trên màn hình

Alt+F5 : Nhìn lại kết quả thực hiện chương trình của lần chạy trước đó

F6 : Thay đổi cửa sổ màn hình soạn thảo

Alt +1, 2, 3: Qui định các cửa sổ màn hình 1, 2, 3 trên cùng một trang màn hình

F7 : Thực hiện chương trình theo chế độ từng dòng lệnh kể cả các lệnh trong thân của hàm

F8 : Thực hiện chương trình theo chế độ từng dòng lệnh nhưng coi các lời gọi hàm là một lệnh.

F9 : Dịch chương trình nguồn thành file *.OBJ

CTRL + F9 : Dịch và thực hiện chương trình (đồng thời tạo file *.OBJ sau đó

tạo file mã máy *.EXE)

F10 : Thực hiện trong chế độ thực đơn

Home : Đưa con trỏ về đầu dòng

End : Đưa con trỏ về cuối dòng

PgUp : Đưa con trỏ lên phía trên một trang màn hình

PgDn : Đưa con trỏ xuống phía dưới một trang màn hình

Del : Xoá kí tự tại vị trí con trỏ

Back Space: Xoá kí tự nằm bên trái con trỏ

CTRL+ PgUp : Đưa con trỏ về đầu văn bản

CTRL+ PgDn : Đưa con trỏ về cuối văn bản

Shift + → : Đánh dấu khối văn bản sang bên trái

Shift + ← : Đánh dấu khối văn bản sang bên phải

Shift + ↑ : Đánh dấu khối văn bản theo từng dòng lên phía trên

Shift + ↓ : Đánh dấu khối văn bản theo từng dòng lên phía dưới

CTRL + Y : Xoá cả dòng chứa con trỏ

CTRL+Q+Y : Xoá tới cuối dòng kể từ vị trí con trỏ

CTRL +K+Y : Xoá khối văn bản đã được đánh dấu trước đó

CTRL+K+C : Copy khối văn bản đã được đánh dấu tới vị trí hiện tại của con trỏ

CTRL+K+V : Chuyển khối văn bản đã được đánh dấu tới vị trí hiện tại của con trỏ

CTRL+K+W : Ghi khối đã được đánh dấu lên đĩa. Nếu tên tệp là PRN thì nội dung của nó sẽ được chuyển qua máy in

CTRL+K+R : Đọc một tệp khác từ đĩa vào, phần được đọc coi như một khối được đánh dấu

CTRL +Q+F : Tìm cụm từ đầu tiên xuất hiện trong văn bản

CTRL+Q+A : Tìm và thay thế cụm từ xuất hiện đầu tiên trong văn bản

CTRL+L : Tìm hoặc thay thế từ tiếp theo xuất hiện trong văn bản

Bước 2: Dịch và hiệu chỉnh chương trình (dùng turbo c)

Chúng ta có thể gọi chương trình dịch của C trực tiếp trong chế độ soạn thảo bằng cách bấm phím F9. Chương trình dịch có nhiệm vụ dịch chương trình của người sử dụng từ file chương trình nguồn có phần mở rộng là *.C thành tệp có phần mở rộng là *.OBJ, sau đó liên kết các tệp *.OBJ lại với nhau để tạo nên file chương trình mã máy có dạng *.COM (chương trình mã máy đã được nén) hoặc *.EXE (chương trình mã máy chưa được nén). Quá trình liên kết được thực hiện thông qua trình liên kết Linker.

PTIT

Trong quá trình dịch, chương trình có thể gặp lỗi, có ba loại lỗi chính (không kể tới lỗi do giải thuật). Đó là:

- Lỗi được thông báo bởi từ khoá error (lỗi cú pháp): Loại lỗi này thường xảy ra trong khi soạn thảo chương trình, chúng ta có thể viết sai các từ khoá ví dụ thay vì viết là int chúng ta soạn thảo sai thành Int (lỗi chữ in thường thành in hoa), hoặc viết sai cú pháp các biểu thức như thiếu các dấu ngoặc đơn, ngoặc kép hoặc dấu chấm phảy khi kết thúc một lệnh, hoặc chưa khai báo nguyên mẫu cho hàm.

- Lỗi được thông báo bởi từ khoá Warning (lỗi cảnh báo): Lỗi này thường xảy ra khi ta khai báo biến trong chương trình nhưng lại không sử dụng tới chúng, hoặc lỗi trong các biểu thức kiểm tra khi biến được kiểm tra không xác định được giá trị của nó, hoặc lỗi do thứ tự ưu tiên các phép toán trong biểu thức. Hai loại lỗi error và warning được thông báo ngay khi dịch chương trình thành file *.OBJ. Quá trình liên kết các file *.OBJ để tạo nên file chương trình mã máy *.EXE chỉ được tiếp tục khi chúng ta hiệu đính và khử bỏ mọi lỗi error, còn lỗi warning chỉ là các cảnh báo, chương trình vẫn có thể được dịch và chạy mà không cần sửa các lỗi này. Tuy nhiên, người viết chương trình cũng nên sửa các lỗi warning.

- Loại lỗi thứ ba có thể xảy ra trong quá trình liên kết: Lỗi này thường xuất hiện khi ta sử dụng tới các lời gọi hàm nhưng những hàm đó mới chỉ tồn tại dưới dạng nguyên mẫu (function prototype) mà chưa được mô tả chi tiết các hàm, hoặc những lời hàm gọi chưa đúng với tên của nó. Lỗi này được khắc phục khi ta bổ sung đoạn chương trình con mô tả chi tiết cho hàm hoặc sửa đổi lại những lời gọi hàm tương ứng.

Bước 3: Thực hiện chương trình

Chương trình được thực hiện bằng cách bấm tổ hợp phím CTRL+F9 (thực hiện trong môi trường soạn thảo TC.EXE) hoặc trở về môi trường DOS thực hiện như các chương trình bình thường khác. Nếu kết quả nhận được là sai thì lỗi đó thuộc lỗi thuật toán mà máy tính không thể phát hiện được loại lỗi kiểu này. Để kiểm tra tính đúng đắn của thuật toán, người lập trình thường sử dụng một số bộ giá trị đặc biệt của thông tin vào.

2.2.3 Chương trình đơn giản nhất trong C

Ví dụ: Viết chương trình in ra dòng thông báo "Ngôn ngữ lập trình C".

Trước hết, ta phải tạo ra văn bản chương trình bằng cách soạn thảo sử dụng trình soạn thảo của Turbo C đó là TC.EXE, thông thường được đặt trong thư mục C:\TC\BIN. Trình soạn thảo của Turbo C gần giống với trình soạn thảo của Pascal chỉ khác nhau ở chỗ văn bản chương trình của được ngầm định phần mở rộng là *.C. Trong khi soạn thảo cần chú ý ghi lại chương trình bằng phím F2 hoặc chọn thực đơn File/Save. Trong ví dụ này chúng ta đặt tên là *SmallPrg.c*

Dịch chương trình thành file *SmallPrg.EXE* bằng phím F9, nếu chúng ta muốn vừa dịch và thực hiện chương trình chỉ cần bấm tổ hợp phím CTRL + F9 và xem kết quả đưa ra màn hình. Trong trường hợp gặp lỗi, trình dịch của C sẽ thông báo lỗi để chúng ta chỉnh sửa và hiệu đính lại chương trình. Chương trình hiển thị lên màn hình dòng "Ngôn ngữ lập trình

C" được viết đơn giản như sau:

Ví dụ:

```
#include <conio.h>
/* khai báo việc sử dụng các hàm printf(), getch() trong conio.h*/
void main(void)
{
printf ("Ngôn ngữ lập trình C\ n");/* in ra màn hình*/
getch(); /* lệnh này chờ nhận một kí tự gõ vào*/
}
```

Kết quả thực hiện chương trình: Dòng chữ được in ra

Ngôn ngữ lập trình C

Để tiếp tục hãy bấm tiếp một phím bất kì ta sẽ trở về với trình soạn thảo trong Turbo C.

Chỉ thị #include được gọi là chỉ thị tiền xử lý, có nhiệm vụ liên kết với tệp tương ứng được đặt trong hai kí tự < tên file đầu tệp >. File có dạng *.h được C qui định là các file chứa nguyên mẫu của các hàm và thường được đặt trong thư mục C:\TC\INCLUDE. Như vậy, chỉ thị #include <conio.h> khai báo việc sử dụng các hàm trong file conio.h, trong trường hợp này ta sử dụng hàm printf() và getch().

Một chương trình C, với bất kì kích thước nào, cũng đều bao gồm một hoặc nhiều "hàm", trong thân của hàm có thể là các lệnh hoặc lời gọi hàm, kết thúc một lệnh là kí tự ';'. Các lời gọi hàm sẽ xác định các thao tác tính toán thực tế cần phải thực hiện. Các hàm của C cũng tương tự như các hàm và chương trình con của một chương trình FORTRAN hoặc một thủ tục PASCAL. Trong ví dụ trên main cũng là một hàm như vậy. Thông thường chúng ta được tự do chọn lấy bất kì tên nào để đặt cho hàm, nhưng main là một tên đặc biệt, chương trình sẽ được thực hiện tại điểm đầu của main. Điều này có nghĩa là mọi chương trình trong C phải có một main ở đâu đó. Main sẽ khởi động các hàm khác để thực hiện công việc của nó, một số hàm nằm ở trong văn bản chương trình, một số khác nằm ở các thư viện của các hàm đã viết trước.

Một phương pháp trao đổi dữ liệu giữa các hàm được thực hiện thông qua đối của hàm. Các dấu ngoặc theo sau tên hàm bao quanh danh sách đối. Thông thường, mỗi hàm khi thực hiện đều trả về một giá trị, tuy nhiên cũng có hàm không có giá trị trả về. Kiểu giá trị trả về của hàm được viết đằng trước tên hàm. Nếu không có giá trị trả về thì từ khóa void được dùng để thay thế (main là hàm không có giá trị trả về). Dấu ngoặc nhọn {} bao quanh các câu lệnh tạo nên thân của hàm, chúng tương tự như Begin . . End trong Pascal. Mọi chương trình trong C đều phải được bao trong { } và không có dấu chấm phảy ở cuối văn bản chương trình. Hàm được khởi động thông qua tên của nó, theo sau là danh sách các đối trong ngoặc. Nếu hàm không có đối thì phải viết các dấu ngoặc tròn cho dù trong ngoặc tròn để trống.

Dòng được viết

printf ("Ngôn ngữ lập trình C\ n");

Là một lời gọi tới hàm có tên printf với đối là một hằng xâu kí tự "Ngôn ngữ lập trình C\ n". printf là hàm thư viện để đưa kết quả ra trên màn hình (trừ khi xác định rõ thiết bị nhận là loại gì khác). Trong trường hợp này hàm sẽ cho hiển thị trên màn hình dãy kí tự tạo nên đối.

Dãy các kí tự bất kì nằm trong hai ngoặc kép "...." được gọi là một xâu kí tự hoặc một hằng kí tự. Hiện tại chúng ta chỉ dùng xâu kí tự như là đối của printf và một số hàm khác.

Dãy \ n trong xâu kí tự trên là cú pháp của C để chỉ kí tự xuống dòng, báo hiệu lần đưa ra sau sẽ được thực hiện ở đầu dòng mới. Ngoài ra C còn cho phép dùng \ t để chỉ dấu tab, \ b cho việc lùi lại (backspace), \" cho dấu ngoặc kép, và \ \ cho bản thân dấu sổ chéo.

2.3. Các kiểu dữ liệu cơ sở

Một kiểu dữ liệu (Data Type) được hiểu là tập hợp các giá trị mà một biến thuộc kiểu đó có thể nhận được làm giá trị của biến cùng với các phép toán trên nó. Các kiểu dữ liệu cơ sở trong C bao gồm kiểu các số nguyên (int, long), kiểu số thực (float, double), kiểu kí tự (char). Khác với Pascal, C không xây dựng nên kiểu Boolean, vì bản chất kiểu Boolean là kiểu nguyên chỉ nhận một trong hai giá trị khác 0 hoặc bằng 0.

Biến kiểu char có kích cỡ 1 byte dùng để biểu diễn 1 kí tự trong bảng mã ASCII, thực chất là số nguyên không dấu có giá trị từ 0 đến 255. Chúng ta sẽ còn thảo luận kỹ hơn về kiểu dữ liệu char trong những phần tiếp theo.

Biến kiểu số nguyên có giá trị là các số nguyên và các số nguyên có dấu (âm, dương) int, long int (có thể sử dụng từ khoá signed int, signed long), kiểu số nguyên không dấu unsigned int, unsigned long. Sự khác biệt cơ bản giữa int và long chỉ là sự khác biệt về kích cỡ.

Biến có kiểu float biểu diễn các số thực có độ chính xác đơn.

Biến có kiểu double biểu diễn các số thực có độ chính xác kép.

Sau đây là bảng các giá trị có thể của các kiểu dữ liệu cơ bản của C:

Kiểu	**Miền xác định**	**Kích thước**
char	0.. 255	1 byte
int	-32768 . . 32767	2 byte
long	-2147483648..2147483647	4 byte
unsigned int	0 . . 65535	2 byte
unsigned long	0 . . .2147483647*2=4294967295	4 byte
float	3. 4e-38 . . 3.4e + 38	4 byte
double	1.7e-308 . . 1.7e + 308	8 byte

Toán tử sizeof(tên_kiểu) sẽ cho ta chính xác kích cỡ của kiểu tính theo byte. Chương

trình sau sẽ in ra kích cỡ của từng kiểu dữ liệu cơ bản.

Ví dụ:

```
/* Chương trình kiểm tra kích cỡ các kiểu dữ liệu cơ bản*/
#include      <stdio.h>
#include      <conio.h>
void main(void){
clrscr(); /* hàm xoá toàn bộ màn hình được khai báo trong stdio.h*/
printf("\n Kích cỡ kiểu kí tự: %d", sizeof(char));
printf("\n Kích cỡ kiểu số nguyên: %d", sizeof(int));
printf("\n Kích cỡ kiểu số nguyên dài: %d", sizeof(long));
printf("\n Kích cỡ kiểu số thực: %d", sizeof(float));
printf("\n Kích cỡ kiểu số thực có độ chính xác kép: %d", sizeof(double));
getch();
}
```

2.4. Biến,hằng, câu lệnh và các phép toán

2.4.1 *Biến và hằng*

Biến: Biến là một đại lượng có giá trị thay đổi trong khi thực hiện chương trình. Mỗi biến có một tên và một địa chỉ của vùng nhớ dành riêng cho biến. Mọi biến đều phải khai báo trước khi sử dụng nó. Qui tắc khai báo một biến được thực hiện như sau:

Tên_kiểu_dữ_liệu tên_biến; trong trường hợp có nhiều biến có cùng kiểu, chúng ta có thể khai báo chung trên một dòng trong đó mỗi biến được phân biệt với nhau bởi một dấu phẩy và có thể gán giá trị ban đầu cho biến trong khi khai báo.

Ví dụ :

```
int a, b, c=0;
/* khai báo 3 biến a, b, c có kiểu int trong đó c được gán là 0*/
float e, f, g= 1.5; /* khai báo 3 biến e, f, g có kiểu float*/
long i, j; /* khai báo i, j có kiểu long*/
unsigned k,m; /* khai báo k,m có kiểu số nguyên dương*/
char key; /* khai báo key có kiểu char*/
```

- **Hằng :** Hằng là đại lượng mà giá trị của nó không thay đổi trong thời gian thực hiện chương trình. C sử dụng chỉ thị #define để định nghĩa các hằng.

- Hằng có giá trị trong miền xác định của kiểu int là hằng kiểu nguyên (nếu không có l ở cuối).

- Hằng có giá trị trong miền xác định của kiểu int và có kí hiệu 0x ở đầu là hằng kiểu nguyên biểu diễn theo cơ số hệ 16 (0xFF).

- Hằng có giá trị trong miền xác định của kiểu long và có kí hiệu L (l) ở cuối cũng

được coi là hằng kiểu long (135L).

- Hằng có giá trị trong miền xác định của kiểu long là hằng kiểu long
- Hằng có giá trị trong miền xác định của kiểu float là hằng kiểu số thực (3.414).
- Hằng có giá trị là một kí tự được bao trong dấu nháy đơn được gọi là hằng kí tự ('A').
- Hằng có giá trị là một dãy các kí tự được bao trong dấu nháy kép được gọi là hằng xâu kí tự "Hằng String".

Ví dụ:

```
#define     MAX     100 /* định nghĩa hằng kiểu nguyên*/
  #define   MIN     0xFF /* hằng nguyên biểu diễn theo cơ số hệ 16*/
  #define   N       123L /* hằng long*/
  #define   PI      3.414   /* hằng thực*/
  #define   KITU    'A'     /* hằng kí tự */
  #define   STR     "XAU KI TU" /*hằng xâu kí tự*/
```

2.4.2 Câu lệnh

Câu lệnh là phần xác định công việc mà chương trình phải thực hiện để xử lý các dữ liệu đã được mô tả và khai báo. Trong C các câu lệnh cách nhau bởi dấu chấm phẩy. Câu lệnh được chia ra làm hai loại: câu lệnh đơn giản và câu lệnh có cấu trúc

Câu lệnh đơn giản là lệnh không chứa các lệnh khác như lệnh gán; lệnh gán được dùng để gán giá trị của biểu thức, một hằng vào một biến, phép gán được viết tổng quát như sau: biến= biểu thức.

Câu lệnh có cấu trúc: Bao gồm nhiều lệnh đơn giản và có khi có cả lệnh cáu trúc khác bển trong . Các lệnh loại này như :

+ Cấu trúc lệnh khối (lệnh ghép hay lệnh hợp)
+ Lệnh if
+ Lệnh switch
+ Các lệnh lặp: for, while, do…. while

2.4.3 Các phép toán

- Các phép toán số học:Gồm có: +, -, *, / (cộng, trừ, nhân, chia), % (lấy phần dư). Phép chia (/) sẽ cho lại một số nguyên giống như phép chia nguyên nếu chúng ta thực hiện chia hai đối tượng kiểu nguyên.

Ví dụ:

```
int a=3, b=5, c; /* khai báo ba biến nguyên*/
```

```
float d =3, e=2, f; /* khai báo ba biến thực*/
c = a + b; /* c có giá trị là 8*/
c = a - b; /* c có giá trị là -2*/
c = a / b ; /* c có giá trị là 0*/
c = a % b; /* c có giá trị là 3*/
f = d / e; /* f có giá trị là 1.5*/
```

Để tiện lợi trong viết chương trình cũng như giảm thiểu các kí hiệu sử dụng trong các biểu thức số học. C trang bị một số phép toán tăng và giảm mở rộng cho các số nguyên như sau:

```
a++ ⇔       a = a +1
a--  ⇔      a = a -1
++a ⇔       a = a +1
--a         ⇔       a = a -1
a+=n        ⇔       a = a + n
a-=n        ⇔       a = a - n
a/=n        ⇔       a = a / n
a*=n        ⇔       a = a * n
a%=n        ⇔       a = a % n
```

Chú ý: Mặc dù ++a và a++ đều tăng a lên một đơn vị, nhưng khi thực hiện các biểu thức so sánh, ++a sẽ tăng a trước rồi thực hiện so sánh, còn a++ sẽ so sánh trước sau đó mới tăng a. Tình huống sẽ xảy ra tương tự đối với --a và a--.

Ví dụ 4.3: Kiểm tra lại các phép toán số học trên hai số nguyên a và b;

```
#include      <stdio.h>
#include      <conio.h>
void main(void)
{
int    a=5, b=2;
clrscr();
printf("\ tổng a + b = %d", a + b);
printf("\ hiệu a - b = %d", a - b);
printf("\ tích a * b = %d", a * b);
printf("\ thương a / b = %d", a / b);
/* thương hai số nguyên sẽ là một số nguyên*/
printf("\ phần dư a % b = %d", a % b);
a++; b--; /* a = a +1; b= b-1; */
printf("\n giá trị của a, b sau khi tăng (giảm): a =%d b =%d", a, b);
```

PTIT

```
a+=b;          /* a=a+b;*/
printf("\n giá trị của a sau khi tăng b đơn vị: a =%d", a);
a-=b;                  /* a = a - b*/
printf("\n giá trị của a sau khi trừ b đơn vị: a =%d", a);
a*=b;                  /* a = a*b;*/
printf("\n giá trị của a sau khi nhân b đơn vị: a =%d", a);
a/=b;                  /* a= a/b; */
printf("\n giá trị của a sau khi chia b đơn vị: a =%d", a);
a %=b;         /* a = a %b; */
printf("\n giá trị của a sau khi lấy modul b : a =%d", a);
getch();
}
```

- Các phép toán so sánh: Gồm có các phép >, <, >=, <=, ==, != (lớn hơn, nhỏ hơn, lớn hơn hoặc bằng, nhỏ hơn hoặc bằng, đúng bằng, khác).\

Ví dụ:

```
if ( a>b) { . . } /* nếu a lớn hơn b*/
if ( a>=b) { . . } /* nếu a lớn hơn hoặc bằng b*/
if ( a==b) { . . } /* nếu a đúng bằng b*/
if ( a!=b) { . . } /* nếu a khác b*/
```

- Các phép toán logic

&& : Phép và logic chỉ cho giá trị đúng khi hai biểu thức tham gia đều có giá trị đúng (giá trị đúng của một biểu thức trong C được hiểu là biểu thức có giá trị khác 0).

|| : Phép hoặc logic chỉ cho giá trị sai khi cả hai biểu thức tham gia đều có giá trị sai.

! : Phép phủ định cho giá trị đúng nếu biểu thức có giá trị sai và ngược lại cho giá trị sai khi biểu thức có giá trị đúng.

Ví dụ:

```
int a =3, b =5;
if ( (a !=0) && (b!=0) ) /* nếu a khác 0 và b khác 0*/
if ((a!=0) || (b!=0)) /* nếu a khác 0 hoặc b khác 0*/
if ( !(a) )     /* phủ định a khác 0*/
```

- Các toán tử thao tác bít:

Các toán tử thao tác bít không sử dụng cho float và double:

& : Phép và (and) các bít.

| : Phép hoặc (or) các bít.

^ : Phép hoặc loại trừ bít (*XOR*).

<< : Phép dịch trái (dịch sang trái n bít giá trị 0)

>> : Phép dịch phải (dịch sang phải n bít có giá trị 0)

~ : Phép lấy phần bù.

Ví dụ:

Giả sử $(a)_{10}$ =3, $(b)_{10}$=5 khi đó $(c)_{10}$ = a & b cho ta kết quả là 1:

```
                0000.0000.0000.0011          a=3
    &           0000.0000.0000.0101          (b)=5
                0000.0000.0000.0001          c =1
```

c = a | b; cho ta kết quả là 7;

```
                0000.0000.0000.0011          a =3
      |         0000.0000.0000.0101          b =5
                0000.0000.0000.0111          c =7
```

c = a ^ b; cho ta kết quả là 6;

```
            0000.0000.0000.0011          a =3
    ^       0000.0000.0000.0101          b=5
            0000.0000.0000.0110          c =6
```

c = ~a; cho ta kết quả là 65532;

```
            ~   0000.0000.0000.0011          a =3
                1111.1111.1111.1100          c = 65532
```

c = a<<2; cho ta kết quả là $(0000.0000.0000.1100)_2$

c=a>>1; cho ta kết quả là $(0000.0000.0000.0001)_2$

- Toán tử chuyển đổi kiểu :

Ta có thể dùng toán tử chuyển đổi kiểu để nhận được kết quả tính toán như mong muốn. Qui tắc chuyển đổi kiểu được thực hiện theo qui tắc: (kiểu) biến

Ví dụ: Tính giá trị phép chia hai số nguyên a và b.

```
#include      <stdio.h>
{
int a=3, b=5;
float c;
c= (float) a / (float) b;
printf("\n thương c = a / b =%6.2f", c);
getch();
}
```

- Thứ tự ưu tiên các phép toán

Khi viết một biểu thức, chúng ta cần lưu ý tới thứ tự ưu tiên tính toán các phép toán, các bảng tổng hợp sau đây phản ánh trật tự ưu tiên tính toán của các phép toán số học và

phép toán so sánh.

Bảng tổng hợp thứ tự ưu tiên tính toán các phép toán số học và so sánh

Tên toán tử	Chiều tính toán
( ), [] , ->	Trái -> Phải
- , ++, -- , ! , ~ , sizeof()	Phải -> Trái
* , /, %	Trái -> Phải
+ , -	Trái -> Phải
>>, <<	Trái -> Phải
<, <=, > , >=,	Trái -> Phải
== !=	Trái -> Phải
&	Trái -> Phải
^	Trái -> Phải
|	Trái -> Phải
&&	Trái -> Phải
||	Trái -> Phải
?:	Phải -> Trái
=, +=, -=, *=, /=, %=, &=, ^=, |=, <<=, >>=	Phải -> Trái

2.5. Thủ tục vào và ra chuẩn

2.5.1 Vào ra ra bằng getchar(), putchar()

Cơ chế vào đơn giản nhất là đọc từng kí tự từ thiết bị vào chuẩn (bàn phím, màn hình) bằng getchar. Mỗi khi được gọi tới getchar() sẽ cho kí tự đọc vào tiếp theo. getchar cho giá trị EOF khi nó gặp cuối tệp trên bất cứ cái vào nào đang được đọc. Thư viện chuẩn định nghĩa hằng kí hiệu EOF là -1 (với #define trong tệp stdio.h) nhưng các phép kiểm tra phải viết dưới dạng EOF chứ không là -1 để cho độc lập với giá trị cụ thể.

Để đưa ra, putchar(c) sẽ đặt kí tự trên "thiết bị ra chuẩn", cũng có giá trị mặc định là màn hình.

Việc đưa ra cho printf cũng chuyển tới thiết bị ra chuẩn, các lời gọi tới putchar và printf có thể chen lẫn nhau.

Nhiều chương trình chỉ đọc trên một thiết bị vào và viết trên một thiết bị ra; với việc vào và ra của các chương trình thì sử dụng getchar, putchar kết hợp với printf là hoàn toàn thích hợp và đầy đủ. Điều này đặc biệt đúng khi làm việc với tệp và sử dụng công cụ đường ống nối đầu ra của chương trình này thành đầu vào của chương trình tiếp. Chẳng hạn, xét chương trình lower, chuyển kí tự vào của nó thành chữ thường:

PTIT

```
#include <stdio.h>
void main() /*chuyển kí tự vào thành chữ thường*/
{
    int c;
while ((c = getchar()) ! = EOF)
 putchar(isupper(c) ? tolower(c) : c);
    }
```

Các "hàm" isupper và tolower thực tế là các macro được xác định trong stdio.h, macro isupper kiểm tra xem đối của nó là chữ hoa hay không, cho giá trị khác 0 nếu đúng như vậy và cho 0 nếu nó là chữ thường. marco tolower chuyển chữ hoa thành chữ thường. Ta không cần biết tới cách các hàm này được cài đặt thế nào trên máy cụ thể, hành vi bên ngoài của chúng như nhau cho nên các chương trình sử dụng chúng không cần để ý tới tập kí tự.

Ngoài ra, trong thư viện vào/ ra chuẩn "các hàm" getchar và putchar là các macro và do vậy tránh được tổn phí về lời gọi hàm cho mỗi kí tự.

2.5.2 *In ra theo khuôn dạng - Printf*

Hai hàm printf để đưa ra và scanf để nhập vào cho phép chuyển ra các biểu diễn kí tự và số. Chúng cũng cho phép sinh ra hoặc thông dịch các dòng có khuôn dạng. Trong các chương trước, chúng ta đã dùng printf một cách hình thức mà chưa có những giải thích đầy đủ về nó. Bây giờ chúng ta sẽ mô tả đầy đủ và chính xác hơn cho hàm này.

printf (control, arg1, arg2,...)

printf chuyển, tạo khuôn dạng, và in các đối của nó ra thiết bị ra chuẩn dưới sự điều khiển của xâu control. Xâu điều khiển của control đều được đưa vào bằng kí tự % và kết thúc bởi một kí tự chuyển dạng. Giữa % và kí tự chuyển dạng có thể có. Dấu trừ (-), xác định việc dồn trái cho đối được chuyển dạng trong trường.

Xâu chữ số xác định chiều ngang tối thiểu của trường. Số được chuyển dạng sẽ được in ra trong trường tối thiểu với chiều ngang này, và sẽ rộng hơn nếu cần thiết. Nếu đối được chuyển có ít kí tự hơn là chiều ngang của trường thì nó sẽ được bổ sung thêm kí tự vào bên trái (hoặc phải, nếu có cả chỉ báo dồn trái) để cho đủ chiều rộng trường. Kí tự bổ xung thêm sẽ là dấu trống thông thường hoặc số 0 nếu chiều ngang trường được xác định với số 0 đứng đầu.

Dấu chấm phân tách chiều ngang trường với xâu chữ số tiếp.

Xâu chữ số (độ chính xác) xác định ra số cực đại các kí tự cần phải in ra từ một xâu, hoặc số các chữ số cần phải in ra ở bên phải dấu chấm thập phân của float hay double.

Bộ thay đổi chiều dài l (chữ ell) chỉ ra rằng phần dữ liệu tương ứng là long chứ không phải là int.

Sau đây là các kí tự chuyển dạng và nghĩa của nó là:

d Đối được chuyển sang kí pháp thập phân.

o Đối được chuyển sang kí pháp hệ tám

x Đối được chuyển sang cú pháp hệ mười sáu không dấu(không có 0x đứng trước).

u Đối được chuyển sang kí pháp thập phân không dấu

c Đối được coi là một kí tự riêng biệt.

s Đối là xâu kí tự; các kí tự trong xâu được in cho tới khi gặp kí tự trống hoặc cho tới khi đủ số lượng kí tự được xác định bởi đặc tả về độ chính xác.

e Đối được xem là float hoặc double và được chuyển sang kí pháp thập phân có dạng[-]m.nnnnnnE[+]xx với độ dài của xâu chứa n do độ chính xác xác định. Độ chính xác mặc định là 6.

f Đối được xem là float hoặc double và được chuyển sang kí pháp thập phân có dạng [-]mmm.nnnnn với độ dài của xâu các n do độ chính xác xác định. Độ chính xác mặc định là 6. Lưu ý rằng độ chính xác không xác định ra số các chữ số có nghĩa phải in theo khuôn dạng f.

g Dùng %e hoặc %f, tuỳ theo loại nào ngắn hơn; không in các số không vô nghĩa.

Nếu kí tự đứng sau % không phải là kí tự chuyển dạng thì kí tự đó sẽ được in ra; vậy % sẽ được in ra bởi %%.

Phần lớn các chuyển dạng là hiển nhiên, và đã được minh hoạ ở các chương trước. Một biệt lệ là độ chính xác liên quan tới các xâu. Bảng sau đây sẽ chỉ ra hiệu quả của các loại đặc tả trong việc in “hello, world” (12 kí tự). Chúng ta đặt dấu hai chấm xung quanh chuỗi kí tự in ra để có thể thấy sự trải rộng của nó

:%10s: :hello, world:

:%-10s: :hello, world:

:%20s: :hello, world :

:%-20s: : hello, world:

:%20.10s: :hello, world:

:%-20.10s: :hello, word:

:%.10s :hello,word:

Lưu ý: printf dùng đối thứ nhất của nó để quyết định xem có bao nhiêu đối theo sau và kiểu của chúng là gì. Hàm sẽ bị lẫn lộn và ta sẽ nhận được câu trả lời vô nghĩa nếu không đủ số đối hoặc đối có kiểu sai.

2.5.3 *Nhập vào có khuôn dạng - scanf*

Hàm scanf là hàm tương tự printf, đưa ra nhiều chức năng chuyển dạng như của printf nhưng theo chiều ngược lại.

scanf(control, arg1, arg2,..)

scanf đọc các kí tự từ thiết bị vào chuẩn, thông dịch chúng tương ứng theo khuôn dạng

được xác định trong control, rồi lưu trữ kết quả trong các đối còn lại. Đối điều khiển sẽ được mô tả sau đây; các đối khác đều phải là con trỏ để chỉ ra nơi dữ liệu chuyển dạng tương ứng cần được lưu trữ.

Xâu điều khiển thường chứa các đặc tả chuyển dạng, được dùng để thông dịch trực tiếp dãy vào. Xâu điều khiển có thể chứa:

Dấu cách, dấu tab hoặc dấu xuống dòng (“các kí tự khoảng trắng”), thường bị bỏ qua.

Các kí tự thông thường (khác%) được xem là ứng với kí tự khác khoảng trắng trong dòng vào.

Các đặc tả chuyển dạng, bao gồm kí tự %, kí tự cắt bỏ gán *(tuỳ chọn), một số tuỳ chọn xác định ra chiều ngang cực đại của trường, và một kí tự chuyển dạng.

Đặc tả chuyển dạng điều khiển việc chuyển dạng cho trường vào tiếp theo. Thông thường kết quả sẽ được đặt vào biến được trỏ tới bởi đối tương ứng. Tuy nhiên, nếu việc cắt bỏ gán được nêu ra bởi kí tự * thì trường vào sẽ bị bỏ qua. Trường vào được xác định như một xâu các kí tự khác khoảng trắng và được kéo dài hoặc tới kí tự khoảng trắng tiếp hoặc cho tới khi chiều ngang của trường.

Kí tự chuyển dạng chỉ ra việc thông dịch cho trường vào; đối tương xứng phải là một con trỏ theo yêu cầu của lời gọi bởi ngữ nghĩa giá trị của C. Các kí tự chuyển dạng sau đây là hợp pháp:

d nhận một số nguyên trong trường vào; đối tương ứng phải là con trỏ nguyên.

o nhận số nguyên hệ tám trong trường vào (có hoặc không có số không đứng trước) đối tương ứng phải là con trỏ nguyên.

x nhận số nguyên hệ mười sáu trong vào (có hoặc không có 0x đứng trước); đối tương ứng phải là con trỏ nguyên.

h nhận số nguyên short trong trường vào; đối tương ứng phải là con trỏ nguyên short.

c nhận một kí tự; đối tương ứng phải là con trỏ kí tự; kí tự vào tiếp được đặt vào chỗ chỉ ra. Trong trường hợp này không bỏ qua các kí tự khoảng trắng; để đọc kí tự khác khoảng trắng tiếp tục dùng %1s.

s nhận một xâu kí tự; đối tương ứng phải là con trỏ kí tự trỏ tới bảng các kí tự đủ lớn để nhận được xâu và dấu kết thúc \ 0 sẽ được thêm vào. Xâu kí tự được nhập qua hàm scanf sẽ không nhận khoảng trắng, tab.

f nhận số dấu phẩy động; đối tương ứng phải là con trỏ tới float. Kí tự chuyển dạng e đồng nghĩa với f. Khuôn dạng vào cho float là một dấu tuỳ chọn, một xâu các số có thể chứa dấu chấm thập phân và một trường mũ tuỳ chọn chứa E hoặc e theo sau là một số nguyên có dấu.

Có thể viết thêm l (chữ ell) vào trước kí tự chuyển dạng d, o và x để chỉ ra rằng con trỏ tới long chứ không phải là int sẽ xuất hiện trong danh sách đối. Tương tự, có thể đặt l vào trước các kí tự chuyển dạng e hoặc f để chỉ ra rằng con trỏ trỏ tới double chứ không trỏ tới float trong danh sách đối.

Chẳng hạn, lời gọi

```
int i;
 float x;
char name[50];
scanf (“%d %f %s”, &i, &x, name);
```

Với dòng vào

25 54.32 E-1 Thompson

Sẽ gán giá trị 25 cho i, giá trị 5.432 cho x và xâu “Thompson” có cả dấu kết thúc \ 0, cho name. Ba trường vào có thể cách nhau bởi nhiều dấu cách, dấu tab và dấu xuống dòng.

Lời gọi

```
int i;
float x;
char name[50];
scanf(“%2d %f %*d %2s”, &i, &x, name);
```

Với đầu vào

56_789 _0123 45a72

Sẽ gán 56 cho i, gán 789.0 cho x, nhảy qua 0123 và đặt xâu “45” vào name. Lời gọi tiếp tới bất kì trình vào nào cũng sẽ bắt đầu tìm tới chữ a. Trong hai ví dụ trên, name là con trỏ và do vậy phải không được đứng sau &.

Xét ví dụ khác, chương trình bàn tính thô sơ có thể được viết với scanf để thực hiện chuyển dạng cái vào

```
#include <stdio.h>
main() /*bàn tính thô sơ*/
{
double sum, v;
sum = 0;
while(scanf(“%1f”, &v) ! = EOF)
        printf("\ t%.2f \ n", sum + = v);
}
```

scanf dừng lại khi nó vét hết xâu điều khiển hoặc khi dữ liệu vào nào đó không sánh đúng với đặc tả điều khiển. Hàm này cho giá trị là số các khoản mục vào đã được đối sánh đúng và được gán. Do vậy có thể dùng giá trị của hàm để xác định xem đã tìm thấy bao nhiêu dữ liệu vào. Khi gặp dấu hiệu cuối tệp hàm sẽ cho giá trị EOF, lưu ý rằng giá trị này khác 0, giá trị 0 có nghĩa là kí tự vào tiếp sẽ không đối sánh đúng với đặc tả đầu tiên trong xâu điều khiển. Lời gọi tiếp tới scanf sẽ tìm đọc tiếp ngay sau kí tự cuối cùng vừa cho.

Điều cần lưu ý nhất là: các đối của scanf phải là con trỏ. Lỗi hay mắc nhất là viết scanf (“%d”, n);Đúng ra phải là scanf(“%d”, &n);

Tương tự như các hàm scanf và printf là sscanf và sprintf, thực hiện các phép chuyển dạng tương ứng nhưng làm việc trên các xâu chứ không trên tệp. Khuôn dạng tổng quát

của chúng là:

sprintf(string, control, arg1, arg2,..)
sscantf(string, control, arg1, arg2,..)

sprintf tạo khuôn dạng cho các đối trong arg1, arg2, v.v... tương ứng theo control như trước, nhưng đặt kết quả vào string chứ không đưa ra thiết bị chuẩn. Tất nhiên string phải đủ lớn để nhận kết quả. Ví dụ: nếu name là một bảng kí tự và n là nguyên thì

sprintf (name, “temp%d”, n);

Tạo ra một xâu có dạng tempnnn trong name với nnn là giá trị của n.

scanf làm công việc ngược lại - nó nhòm vào string tương ứng theo khuôn dạng trong control và đặt các giá trị kết quả vào arg1, arg2... Các đối phải là con trỏ. Lời gọi

sscanf(name, “temp%d”, n);

đặt cho n giá trị của xâu các chữ số đứng sau temp trong name.

2.5.4 Thâm nhập vào thư viện chuẩn

Mỗi tệp gốc có tham trỏ tới hàm thư viện chuẩn đều phải chứa dòng khai báo

#include< tên_tệp_thư_viện >

ở đầu tệp văn bản chương trình. Dấu ngoặc nhọn < tên_tệp_thư_viện > thay cho dấu nháy thông thường để chỉ thị cho trình biên dịch tìm kiếm tệp trong danh mục chứa thông tin tiêu đề chuẩn được lưu trữ trong thư mục include. Trong trường hợp chúng ta sử dụng kí tự “tên_tệp_thư_viện” trình biên dịch sẽ tìm kiếm tệp thư viện tại thư mục hiện tại.

Chỉ thị #include “tên_tệp_thư_viện” còn có nhiệm vụ chèn thư viện hàm của người sử dụng vào vị trí của khai báo. Trong ví dụ sau, chúng ta sẽ viết chương trình thành 3 tệp, tệp define.h dùng để khai báo tất cả các hằng sử dụng trong chương trình, tệp songuyen.h dùng khai báo nguyên mẫu của hàm và mô tả chi tiết các hàm giống như một thư viện của người sử dụng, tệp mainprg.c là chương trình chính ở đó có những lời gọi hàm từ tệp songuyen.h.

Ví dụ: Xây dựng một thư viện đơn giản mô tả tập thao tác với số nguyên bao gồm: tính tổng, hiệu, tích, thương, phần dư, ước số chung lớn nhất của hai số nguyên dương a, b.

/* Nội dung tệp define.h */

```
#include    <stdio.h>
#include    <conio.h>
#include    <dos.h>
#define     F1      59
#define     F2      60
#define     F3      61
#define     F4      62
#define     F5      63
#define     F6      64
#define     F1      65
```

```
#define      F10     68
/* Nội dung tệp songuyen.h */
void Init_Int( int *, int *);
int      Tong_Int(int , int );
int      Hieu_Int(int, int );
int      Tich_Int(int, int );
float    Thuong(int , int );
int      Mod_Int(int, int);
int      UCLN(int , int);
void     Init_Int( int *a, int *b ){
         printf("\n Nhập a = "); scanf( "%d", a);
         printf("\n Nhập b = "); scanf( "%d", b);
}
int      Tong_Int( int a, int b ){
         printf("\n Tổng a + b =%d", a + b);
         delay(2000);
         return(a+b);
}
int      Hieu_Int( int a, int b ){
         printf("\n Hiệu a - b =%d", a - b);
         delay(2000);
         return(a - b);
}
int      Tich_Int( int a, int b ){
         printf("\n Tích a * b =%d", a * b);
         delay(2000);
         return(a*b);
}
float    Thuong_Int( int a, int b ){
         printf("\n Thương a / b =%6.2f", (float) a /(float) b);
         delay(2000);
         return((float) a /(float) b);
}
int      Mod_Int( int a, int b ){
         printf("\n Phần dư a % b =%d", a % b);
         delay(2000);
         return(a%b);
}
int      UCLN( int a, int b) {
         while (a != b) {
                  if ( a > b) a -=b;
```

PTIT

```
            else    b-=a;
        }
        printf("\n UCLN =%d", a);
        delay(2000); return(a);
}
```

/* Nội dung tệp mainprg.c */

```
#include    "define.h"
#include    "songuyen.c"
void thuchien(void){
    int a, b, control = 0; char c; textmode(0);
    do {
            clrscr();
            printf("\n Tập thao tác với số nguyên");
            printf("\n F1- Nhập hai số nguyên");
            printf("\n F2- Tổng hai số nguyên");
            printf("\n F3- Hiệu hai số nguyên");
            printf("\n F4- Tích hai số nguyên");
            printf("\n F5- Thương hai số nguyên");
            printf("\n F6- Phần dư hai số nguyên");
            printf("\n F7- UCLN hai số nguyên");
            printf("\n F10- Trở về");
            c = getch();
            switch(c) {
                case F1: Init_Int(&a, &b); control =1; break;
                case F2:
                        if (control) Tong_Int(a, b);
                        break;
                case F3:
                        if (control) Hieu_Int(a, b);
                        break;
                case F4:
                        if (control) Tich_Int(a, b);
                        break;
                case F5:
                        if (control) Thuong_Int(a, b);
                        break;
                case F6:
                        if (control) Mod_Int(a, b);
                        break;
                case F7:
                        if (control) UCLN_Int(a, b);
```

PTIT